Experiment No-12

```
import re
import nltk
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS,
TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# Step 1: Sample textual dataset
texts = [
    "I love this product, it is amazing and wonderful",
    "This product is excellent and works perfectly",
    "I am very happy with the quality",
    "The experience was great and I recommend it",
    "Absolutely fantastic service and friendly staff",
    "The product is useful and easy to use",
    "I really enjoyed this excellent experience",
    "This is a good product and worth buying",
    "I hate this product, it is terrible and useless",
    "The quality is poor and disappointing",
    "I am unhappy with this bad service",
    "The experience was awful and I do not recommend it",
    "Very frustrating, the product broke quickly",
    "This is a horrible product and a waste of money",
    "I regret buying it, the quality is bad",
    "Terrible service and unfriendly staff"
]

labels = [
    "positive", "positive", "positive", "positive",
    "positive", "positive", "positive", "positive",
    "negative", "negative", "negative", "negative",
```

```
    "negative", "negative", "negative", "negative"
]

# Step 2: Initialize the stemmer and stop words
stemmer = PorterStemmer()
stop_words = set(ENGLISH_STOP_WORDS)

# Step 3: Text preprocessing
def preprocess(text):
    text = text.lower()
    words = re.findall(r"\b[a-z]+\b", text)

    # Remove stop words and apply stemming
    processed_words = [
        stemmer.stem(word)
        for word in words
        if word not in stop_words
    ]

    return " ".join(processed_words)

# Display preprocessing results
print("TEXT PREPROCESSING")
for text in texts[:3]:
    print("\nOriginal:", text)
    print("Processed:", preprocess(text))

# Step 4: Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)

# Step 5: Convert text into numerical features
```

```
vectorizer = TfidfVectorizer(
    preprocessor=preprocess,
    token_pattern=r"(?u)\b\w+\b"
)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# Step 6: Train the sentiment classifier
model = LogisticRegression(max_iter=1000)
model.fit(X_train_tfidf, y_train)

# Step 7: Predict sentiment
predictions = model.predict(X_test_tfidf)

print("\nMODEL EVALUATION")
print("Accuracy:", round(accuracy_score(y_test, predictions) * 100, 2),
"%")
print(classification_report(
    y_test, predictions, zero_division=0
))

# Step 8: Test with new sentences
new_texts = [
    "The product is amazing and I love it",
    "The service was terrible and disappointing",
    "The quality is excellent"
]

print("\nSENTIMENT PREDICTIONS")
new_features = vectorizer.transform(new_texts)
new_predictions = model.predict(new_features)

for text, sentiment in zip(new_texts, new_predictions):
    print(f"Text: {text}")
    print(f"Sentiment: {sentiment.upper()}\n")
```

Output of the code:-

```
TEXT PREPROCESSING

Original: I love this product, it is amazing and wonderful
Processed: love product amaz wonder

Original: This product is excellent and works perfectly
Processed: product excel work perfectli

Original: I am very happy with the quality
Processed: happi qualiti

MODEL EVALUATION
Accuracy: 25.0 %
              precision    recall  f1-score   support

    negative       0.00      0.00      0.00         2
    positive       0.33      0.50      0.40         2

    accuracy                           0.25         4
   macro avg       0.17      0.25      0.20         4
weighted avg       0.17      0.25      0.20         4


SENTIMENT PREDICTIONS
Text: The product is amazing and I love it
Sentiment: POSITIVE

Text: The service was terrible and disappointing
Sentiment: NEGATIVE

Text: The quality is excellent
Sentiment: POSITIVE
```